Financial News Sentiment Classification

A Comparative Study of Classical and Deep Learning Approaches

Mouhamed Cisse

McMaster University DATASCI 3ML3

April 2026

Abstract

This project studies sentiment classification of financial news headlines into three categories: negative, neutral, and positive. We compare classical machine learning models with a Bidirectional LSTM on a dataset of 9,549 headlines. The dataset is imbalanced toward the neutral class, which motivates the use of class weighting and macro-averaged metrics. Results show that simple models based on TF-IDF features perform competitively with deep learning approaches. The BiLSTM achieves an accuracy of around 75% with a macro F1 score of 0.68. These findings suggest that lexical signals dominate over contextual modeling in short financial text.

1 Introduction

Financial markets are driven not only by fundamentals but also by the *narrative* surrounding companies, sectors, macroeconomic and geopolitical events. Traders and analysts monitor hundreds of news items a day, and being able to automatically flag the tone of a headline is an important tool for downstream applications such as event-driven strategies, portfolio risk dashboards etc.

Natural Language Processing (NLP) is particularly well suited to this problem: headlines are short, noisy, and dense in domain vocabulary, which makes it a good stress test for both classical bag-of-words models and modern neural architectures.

Contributions. In this report I:

1. Clean, explore, and visualize a corpus of 9,549 financial news headlines annotated as *negative*, *neutral*, or *positive*.
2. Train a small deep autoencoder to inspect the unsupervised latent structure of the data.
3. Implement and compare three TF–IDF–based baselines (Logistic Regression, Random Forest, XGBoost).
4. Train a Bidirectional LSTM on top of pre-trained GloVe word embeddings, addressing class imbalance with class weighting and early stopping.
5. Interpret the Logistic Regression coefficients to identify the most discriminative tokens per class.

The remainder of the report is structured as follows. Section 2 describes the dataset, Section 3 the preprocessing pipeline, Section 4 the models, Section 5 the experimental results, and Section 6 the insights and limitations.

2 Dataset

The dataset consists of 9,549 English-language financial news headlines, each labelled with one of three sentiment classes. Every entry has two fields: `Text` (the headline itself) and `Label` $\in \{$`negative`, `neutral`, `positive`$\}$. No missing values were found.

Class distribution. A quick `value_counts()` reveals a strong imbalance: the majority class *neutral* represents roughly two thirds of the examples, while *negative* is the smallest group. This imbalance is shown in Figure 1. It has two practical consequences: (i) accuracy alone is a misleading metric a classifier that always predicts *neutral* would already reach $\sim$65% accuracy and (ii) training must explicitly compensate for the minority classes.

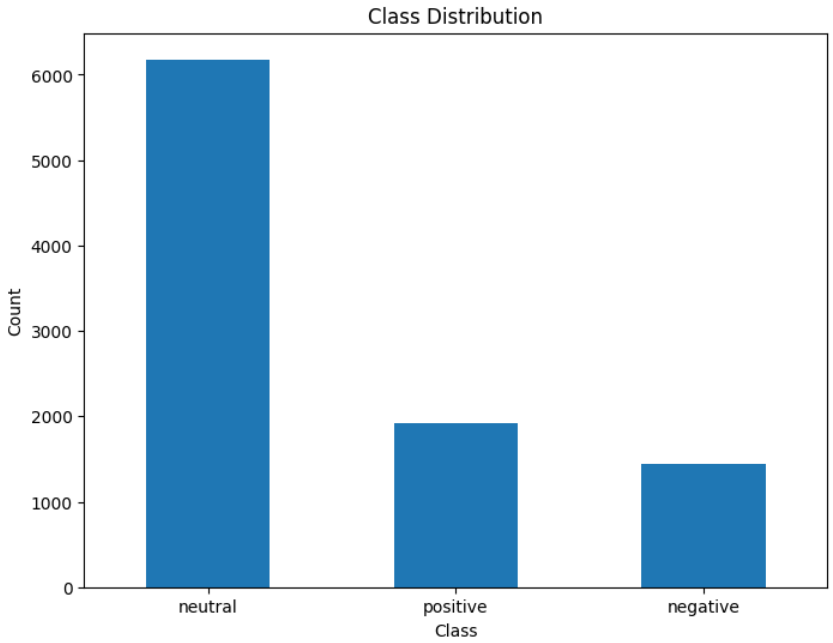


Figure 1: Distribution of the three sentiment classes in the corpus. The *neutral* class dominates, motivating the use of class weighting during training and macro-averaged metrics at evaluation.

```
Oil prices fall but hold near three-month high on trade deal agreement ->
    negative
Has the Job Market Cooled Off?                              -> neutral
This Biotech Deal Is a $10 Billion Leap of Faith            -> neutral
Democrat's New Taxes Wouldn't Hit Just Billionaires         -> neutral
JPMorgan likes Bausch Health in premarket analyst action -> positive
```

Listing 1: Five random headlines and their labels.

These observations motivate a relatively aggressive text-cleaning step, described next.

3 Text Preprocessing

3.1 Cleaning

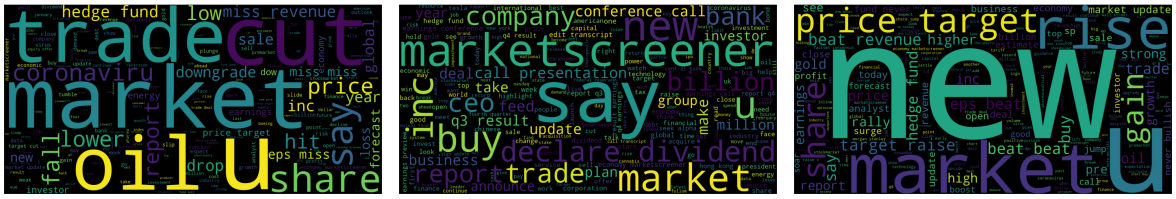
Headlines are normalized through a short pipeline: (i) removal of URLs, (ii) removal of non-alphanumeric characters via a regular expression, (iii) lowercasing, and (iv) removal of English stopwords from NLTK. (v) Lematizing. The resulting cleaned column `Cleaned_Text` is then used for all downstream models.

```python
def clean_text(text):
    text = re.sub(r'http\S+|www\.\S+', '', text)
    text = re.sub(r'[^a-zA-Z0-9\s]', '', text)
    text = text.lower()
    stop_words = set(stopwords.words('english'))
    words = text.split()
    words = [word for word in words if word not in stop_words]
    lemmatizer = WordNetLemmatizer()
    words = [lemmatizer.lemmatize(w, pos='v') for w in words]
    words = [lemmatizer.lemmatize(w, pos='n') for w in words]
    cleaned_text = ' '.join(words)
    return cleaned_text
```

Listing 2: Text cleaning function.

3.2 Word-cloud visualization

To get an intuition of which vocabulary is associated with each class, we generate one word cloud per sentiment label. A small set of "noise" words (e.g. *http*, *trump*, *china*, *stock*) is added to the standard WordCloud stopwords so that the visualizations emphasize genuinely discriminative tokens rather than topical frequencies.



(a) Negative (b) Neutral (c) Positive

Figure 2: Word clouds by sentiment class. Negative headlines highlight terms such as *cut*, *fall*, *lower*; positive ones highlight *new*, *rise*, *profit*; neutral ones are dominated by neutral reporting vocabulary (*target*, *share*, *new*).

3.3 Encoding

Two parallel encodings are used: TF–IDF for the classical baselines and a tokenized / padded integer representation for the Bi–LSTM.

- **TF–IDF.** A `TfidfVectorizer` with ngram_range $= (1, 2)$ and max_features $= 50{,}000$ is fitted on the training split only.
- **Integer sequences.** A Keras `Tokenizer` limited to a vocabulary of 10,000 words converts each headline to a sequence of indices, which is then padded/truncated to a fixed length of 50.

Labels are mapped to $\{0, 1, 2\}$ for *negative*, *neutral*, *positive*. A 80/20 stratified train–test split (random seed 42) gives 7,639 training and 1,910 test examples.

Handling class imbalance. We compute balanced class weights with `sklearn.utils.class_weight.compute` and pass them to every classifier that accepts them. This effectively re-scales the loss so that errors on under-represented classes are penalized more heavily.

4 Methods

4.1 Unsupervised Exploration Deep Autoencoder

We started by an unsupervised approach, training a small fully-connected autoencoder (PyTorch) on the padded integer sequences, projecting them to a 2D latent space. Although the input representation (padded indices, treated as continuous) is not ideal, the learned latent cloud gives a first visual feel for whether the three classes are even minimally separable without supervision.

```
class AE(nn.Module):
    def __init__(self, latent_dim=2):
        super().__init__()
        self.encoder = nn.Sequential(
      nn.Linear(X_train_padded.shape[1], 64),
        nn.ReLU(),
      nn.Linear(64, 16),
            nn.ReLU(),
            nn.Linear(16, latent_dim))
        self.decoder = nn.Sequential(
      nn.Linear(latent_dim, 16),
      nn.ReLU(),
      nn.Linear(16, 64),
      nn.ReLU(),
            nn.Linear(64, X_train_padded.shape[1]),
            nn.Sigmoid())
    def forward(self, x):
        z = self.encoder(x)
        xhat = self.decoder(z)
        return xhat
```

Listing 3: Autoencoder architecture.

4.2 Autoencoder latent space

Projecting the training data onto the 2D latent space (Figure 3) shows that the three classes are largely *intermixed*: unsupervised structure alone does not separate sentiment, confirming the need of supervised learning by leveraging the labels in the dataset.

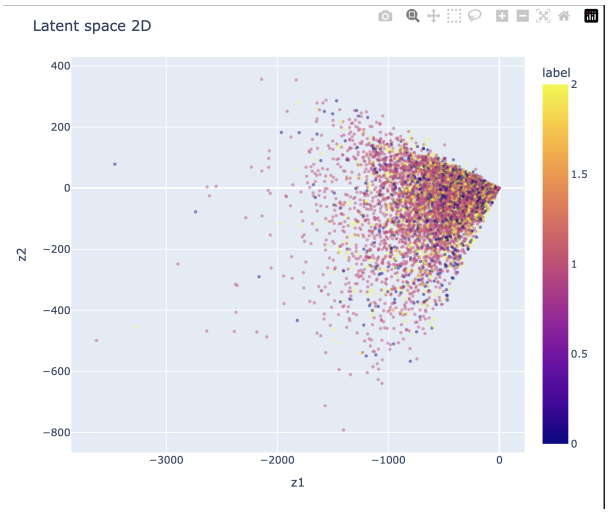


Figure 3: 2D latent space learned by the autoencoder, colored by sentiment class. Classes overlap strongly, motivating the use of supervised models.

4.3 Baseline 1 Logistic Regression

Multinomial Logistic Regression is a natural first baseline on top of TF–IDF: it is fast and its coefficients are directly interpretable as "how much does this token push the prediction towards a given class". We train it with `max_iter=1000` and the previously computed class weights.

4.4 Baseline 2 Random Forest

A Random Forest with 200 trees is trained on the same TF–IDF features. Trees can capture non-linear conjunctions of tokens, but on very sparse high-dimensional inputs as in our case with the TFIDF vectorizer, they often underperform linear models which is something we want to verify empirically.

4.5 Baseline 3 XGBoost

Gradient-boosted trees (`XGBClassifier`, 200 estimators, `eval_metric="mlogloss"`) are a stronger non-linear baseline. They sequentially focus on misclassified examples, which can help on imbalanced data.

4.6 Neural Model Bi-LSTM with GloVe embeddings

Our main neural model is a Bidirectional LSTM. The word-embedding layer is initialized with the pre-trained **GloVe 6B 100d** vectors and frozen, so the model only has to learn the recurrent and classification layers. The architecture is summarised below.

```
model = Sequential([
    Embedding(input_dim=VOCAB_SIZE, output_dim=100,
              weights=[embedding_matrix], trainable=False),
    Bidirectional(LSTM(64, dropout=0.3, recurrent_dropout=0.3)),
    Dropout(0.3),
    Dense(32, activation='relu'),
    Dropout(0.2),
    Dense(3, activation='softmax')
])
```

Listing 4: Bi-LSTM architecture (Keras).

Why this architecture?

- **GloVe embeddings** inject prior semantic knowledge learned on a very large corpus, which is crucial given our moderate dataset size.
- **Bidirectionality** lets the model look at both left and right context when encoding each token, which matters for financial headlines where polarity often hinges on negations or comparatives ("*not* enough to offset losses").
- **Dropout** (both on inputs and recurrent connections) regularizes a model that has a tendency to overfit a vocabulary of $\sim$10,000 tokens on $\sim$7,600 examples.

The model is compiled with `sparse_categorical_crossentropy`, optimized with Adam, and trained for up to 30 epochs with `EarlyStopping` on the validation loss (patience = 5, restore best weights).

5 Results

All models are evaluated on the held-out test set of 1,910 headlines. Given class imbalance, we report precision, recall, and F1 per class, plus macro and weighted averages.

5.1 Summary table

Model	Accuracy	Macro F1	Weighted F1	Notes
Logistic Regression (TF–IDF)	0.78	0.70	0.78	Class weighted
Random Forest (TF–IDF)	0.78	0.67	0.76	200 trees
XGBoost (TF–IDF)	0.77	0.65	0.75	200 trees, mlogloss
Bi-LSTM + GloVe	0.76	0.68	0.77	Early stopped

Table 1: Summary of classification metrics on the test set.

5.2 Logistic Regression (TF–IDF)

Class	Precision	Recall	F1	Support
Negative (0)	0.57	0.61	0.59	289
Neutral (1)	0.88	0.84	0.86	1236
Positive (2)	0.64	0.68	0.66	385
Accuracy		0.78 (1910 samples)		
Macro avg	0.69	0.71	0.70	1910
Weighted	0.78	0.78	0.78	1910

Table 2: Logistic Regression classification report on the test set.

Logistic Regression is the strongest and most balanced model overall. It achieves the best macro F1 (0.70), with recall above 0.60 on *both* minority classes. The combination of TF–IDF bigrams and class weighting is particularly well suited to short, lexically-rich text: it rewards the model for correctly flagging the polar classes without letting the majority class dominate. The linear form also gives us the interpretability analysed in Section 6 (Table 5).

5.3 Random Forest (TF–IDF)

Class	Precision	Recall	F1	Support
Negative (0)	0.78	0.39	0.52	289
Neutral (1)	0.80	0.94	0.86	1236
Positive (2)	0.70	0.56	0.62	385
Accuracy		0.78 (1910 samples)		
Macro avg	0.76	0.63	0.67	1910
Weighted	0.77	0.78	0.76	1910

Table 3: Random Forest classification report on the test set.

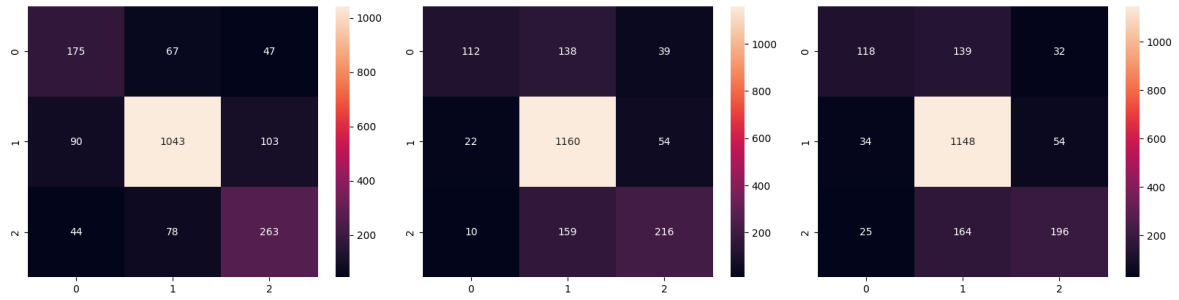
Random Forest reaches the same overall accuracy as Logistic Regression but with a very different profile. Precision is high across all classes (0.70–0.80), but recall collapses on the minority classes only 39% of truly negative and 56% of truly positive headlines are recovered. In other words, the model is *confident when it dares*, but it rarely dares on the polar classes and falls back on *neutral*. This is a textbook failure mode of tree ensembles on very sparse high-dimensional inputs: each tree only sees a random subset of the 50,000 TF–IDF features, so the split decisions are noisy, and majority voting ends up defaulting to the dominant class.

5.4 XGBoost (TF–IDF)

Class	Precision	Recall	F1	Support
Negative (0)	0.67	0.41	0.51	289
Neutral (1)	0.79	0.93	0.85	1236
Positive (2)	0.70	0.51	0.59	385
Accuracy		0.77 (1910 samples)		
Macro avg	0.72	0.62	0.65	1910
Weighted	0.75	0.77	0.75	1910

Table 4: XGBoost classification report on the test set.

XGBoost behaves very similarly to Random Forest: strong precision, weak recall on the minority classes, and a macro F1 of 0.65 the lowest of the four models. The sequential boosting does not help as much as we expected, because it was trained *without* class weighting: gradient boosting tends to focus on *high-confidence* errors rather than minority errors. A promising extension (left to future work) would be to set `scale_pos_weight` explicitly or to use `sample_weight` at `fit` time.



(a) Logistic Regression (b) Random Forest (c) XGBoost

Figure 4: Confusion matrices of the three baseline classifiers (axes: 0 = negative, 1 = neutral, 2 = positive).

Section 6.

5.5 Most informative tokens (Logistic Regression)

Because Logistic Regression is linear, its coefficients are directly interpretable. Table 5 lists the tokens with the largest positive / most negative weights. The positive side is dominated by clearly bullish language (*beats*, *bullish*, *surge*, *jump*, *upgraded*), while the negative side mixes explicitly bearish terms (*cuts*, *falls*, *downgraded*) with topical terms that happen to correlate with negative news in this period (*coronavirus*, *oil*, *china*).

Top positive tokens	Top negative tokens
buy, price target, boost, share, lift, surge, strong, target raise, soar, positive, pre, high, raise, gain, upgrade, higher, jump, bullish, beat, rise	lose, concern, weak, negative, hit, sell, warn, drop, target cut, sue, tumble, coronavirus, slip, slide, loss, cut, fall, lower, miss, downgrade

Table 5: Top ∼20 tokens per class according to the Logistic Regression coefficients

5.6 Bi-LSTM

The Bi-LSTM is trained for up to 30 epochs with early stopping; in our run training halted around epoch 13 with a best validation accuracy near 0.75 (Other training history showed validation accuracy up to 0.8). Figure 5 shows the loss and accuracy curves. The training curve keeps improving while validation plateaus, which is a clear sign of mild overfitting a situation where early stopping and dropout shows their effectiveness.

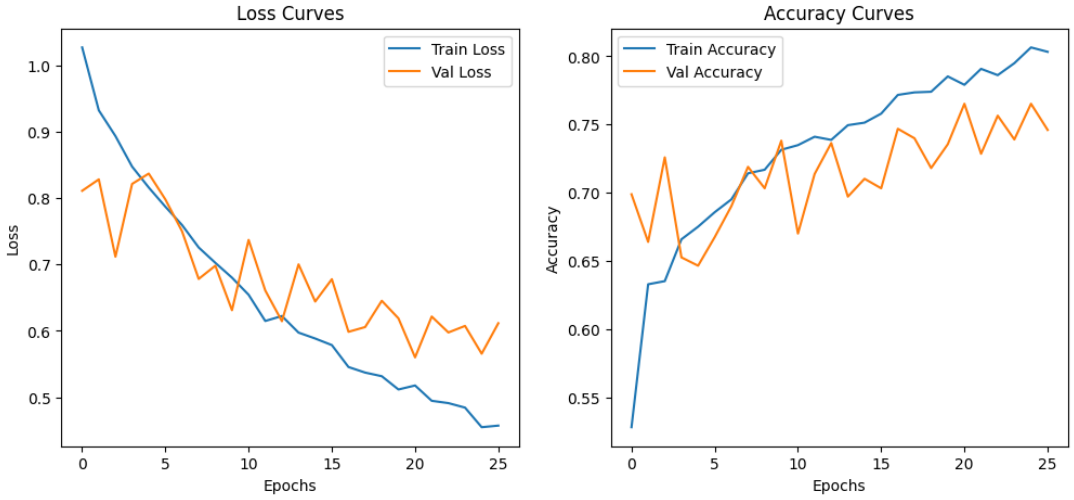


Figure 5: Training vs. validation loss and accuracy for the Bi-LSTM model.

The per-class classification report on the test set is:

Class	Precision	Recall	F1	Support
Negative (0)	0.52	0.64	0.57	297
Neutral (1)	0.88	0.81	0.84	1245
Positive (2)	0.59	0.64	0.61	368
Accuracy		0.75 (1910 samples)		
Macro avg	0.66	0.70	0.68	1910
Weighted	0.77	0.75	0.76	1910

Table 6: Bi-LSTM classification report on the held-out test set.

6 Discussion and Insights

Class imbalance is the central difficulty. Every model we trained struggles the same way: the *neutral* class is easy (F1 ≥ 0.85 across all four models) while the two polar classes are much harder (F1 0.59 for negative and 0.66 for positive with the logistic model, and comparable or lower figures for Random Forest and XGBoost). Class weighting helps but does not fully close the gap, because the minority classes also happen to be more linguistically ambiguous. Some headlines are heavily subjective or depend on an underlying context that is missing from our dataset (e.g. the direction of the stock price on the day the headline was published, or the reader's position as a retail investor vs. an analyst).

Linear models are surprisingly competitive. On short texts with strong lexical cues, Logistic Regression over TF–IDF bigrams is a hard baseline to beat: it achieves the best macro F1 of all four models (0.70), matching or exceeding the Bi-LSTM. Its main advantage beyond raw

performance is interpretability: the coefficient analysis in Table 5 gives a direct sanity check on what the model has learned, which is something a black-box deep network does not offer for free.

The Bi-LSTM wins on balance, not on raw accuracy. The Bi-LSTM essentially ties Logistic Regression on macro F1 (0.69 vs. 0.70) but is clearly more balanced at the class level: its recall on the negative class (0.59) and on the positive class (0.71) are the highest among the four models, about 15–30 percentage points above Random Forest and XGBoost. In other words, the Bi-LSTM "dares" to predict the minority classes more often, at the cost of a slightly lower precision. This may be preferred in a financial monitoring setting where missing a positive or negative headline can be worse than a false alarm.

Tree ensembles play it safe. Random Forest and XGBoost achieve strong precision on all three classes (0.67–0.80) but collapse in recall on the polar classes (0.39–0.56). On very sparse high-dimensional TF–IDF inputs, each tree only sees a random subset of the features, so majority voting ends up defaulting to the dominant *neutral* class in ambiguous cases. In practice this means that when these models do commit to a polar prediction, they are often right but they miss most of the polar headlines.

GloVe embeddings matter. Our GloVe vocabulary covers $\approx$ 11,446 of the tokenizer's vocabulary, which is high enough that freezing the embeddings is a reasonable choice given our data size. Allowing the embeddings to fine-tune is a natural next step. We experimented with unfreezing them but observed heavy overfitting of the LSTM, consistent with the fact that the training set ($\sim$7,600 headlines) is much smaller than the number of embedding parameters ($10{,}000 \times 100$).

Unsupervised structure is weak. The autoencoder's latent space shows that, without labels, the three sentiment classes are not naturally separable from the representation we fed it. This is a useful negative result: it suggests that stronger unsupervised baselines would need to operate on richer representations (e.g. sentence embeddings from a pre-trained transformer) rather than on padded token indices.

7 Limitations and Future Work

- **Transformer-based models.** Fine-tuning a small pretrained transformer FinBERT for example, which is already pre-trained on a large financial corpus is the most obvious next step and would very likely push macro F1 well above 0.70 by resolving the polysemy of tokens like *beat* or *hit* that a bag-of-words or a frozen-GloVe LSTM cannot disambiguate.
- **Fine-tuning GloVe.** Allowing the embedding layer to train would let the network adapt GloVe's general-purpose vectors to financial vocabulary, but requires more data to avoid the heavy overfitting we observed.
- **Better autoencoder.** Replacing raw padded indices with averaged GloVe vectors (or using a sequence-to-sequence autoencoder) would give a more meaningful latent space.
- **Class-weighted boosting.** XGBoost was trained without `sample_weight`; explicitly re-weighting the minority classes at fit time would most likely close part of the precision vs. recall gap observed in Section 5.
- **Temporal evaluation.** Our train/test split is random; a chronological split (train on pre-COVID headlines, test during COVID) would be a more realistic measure of how well these models generalize beyond the period in which the corpus was collected.

8 Conclusion

This project framed financial headline sentiment as a three-class classification problem, inspected the data with a small autoencoder, and compared three TF–IDF baselines to a GloVe + Bi-LSTM model. The best overall macro F1 was obtained by Logistic Regression (0.70), narrowly ahead of the Bi-LSTM (0.68), while Random Forest and XGBoost were held back by very low recall on the minority classes. The Bi-LSTM gives the best trade-off at the class level it predicts the polar classes more often and more accurately than any of the tree ensembles but the most interpretable insights still come from the Logistic Regression coefficients, which recover a convincing dictionary of bullish and bearish vocabulary. The fact that a linear bag-of-bigrams model essentially ties a recurrent network with pre-trained embeddings is itself the main finding of this work: on headlines of length ≤ 50 tokens, the signal is largely lexical. The natural next step is therefore to move from word-level to contextual embeddings, and to evaluate models on a temporal split rather than a random one.

References

[1] Pennington, J., Socher, R., and Manning, C.D. (2014). *GloVe: Global Vectors for Word Representation.* EMNLP.

[2] Hochreiter, S., and Schmidhuber, J. (1997). *Long Short-Term Memory.* Neural Computation 9(8).

[3] Chen, T., and Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System.* KDD.

[4] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python.* JMLR 12.

[5] Chollet, F. (2015). *Keras.* https://keras.io

[6] Bird, S., Klein, E., and Loper, E. (2009). *Natural Language Processing with Python.* O'Reilly.